

Object Detection

Object Detection

Object Detection is a computer vision task where the system:

- **Identifies the object class** (What is the object?)
- **Locates the object** using a bounding box (Where is the object?)
- Unlike simple image classification, object detection can detect **multiple objects** in a single image.

Bounding Box

A **bounding box** is a rectangular box drawn around an object to indicate its location in an image.

A bounding box is represented as:

(x, y, w, h)

Where:

- $x, y \rightarrow$ top-left corner
- $w \rightarrow$ width
- $h \rightarrow$ height

OR

$(x_{\min}, y_{\min}, x_{\max}, y_{\max})$

Example

Suppose an image contains a dog.

The detector predicts:

Left coordinate = 50

Top coordinate = 30

Width = 120

Height = 100

So the bounding box is:

$(x,y,w,h)=(50,30,120,100)$

Where:

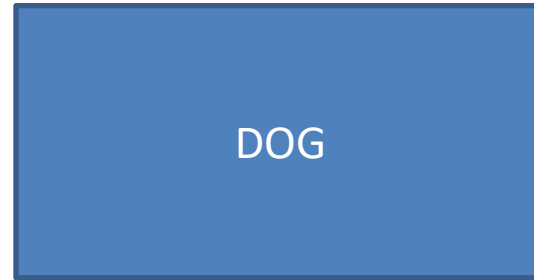
$x=50 \rightarrow$ horizontal position

$y=30 \rightarrow$ vertical position

$w=120 \rightarrow$ width

$h=100 \rightarrow$ height

50,30



Height=100

Width=120

Another Way of Representation:

Bounding boxes are also written as:

$(x_{\min}, y_{\min}, x_{\max}, y_{\max})$

$x_{\max} = 50 + 120 = 170$

$y_{\max} = 30 + 100 = 130$

$(50, 30, 170, 130)$

```
import cv2

# Read image
img = cv2.imread("dog.jpg")

# Bounding box coordinates
x, y, w, h = 50, 30, 120, 100

# Draw rectangle
cv2.rectangle(img, (x, y), (x+w, y+h), (0,255,0),
2)

# Show image
cv2.imshow("Bounding Box", img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

How Object Detection will Achieve

Step 1: Input Image

Image is given to CNN.

Step 2: Feature Extraction

CNN extracts features like:

edges

textures

shapes

Step 3: Region Proposal / Grid Division

Possible object regions are generated.

Step 4: Classification + Localization

For each region:

predict object class

predict bounding box coordinates

Step 5: Non-Max Suppression (NMS)

Remove duplicate overlapping boxes.

Intersection over Union (IoU)

Intersection over Union (IoU) is a metric used in object detection to measure how well the **predicted bounding box** matches the **actual (ground truth) bounding box**.

$$IoU = \frac{\text{Area of Overlap}}{\text{Area of Union}}$$

It is needed to:

- Predict the object class
- Predict the correct location

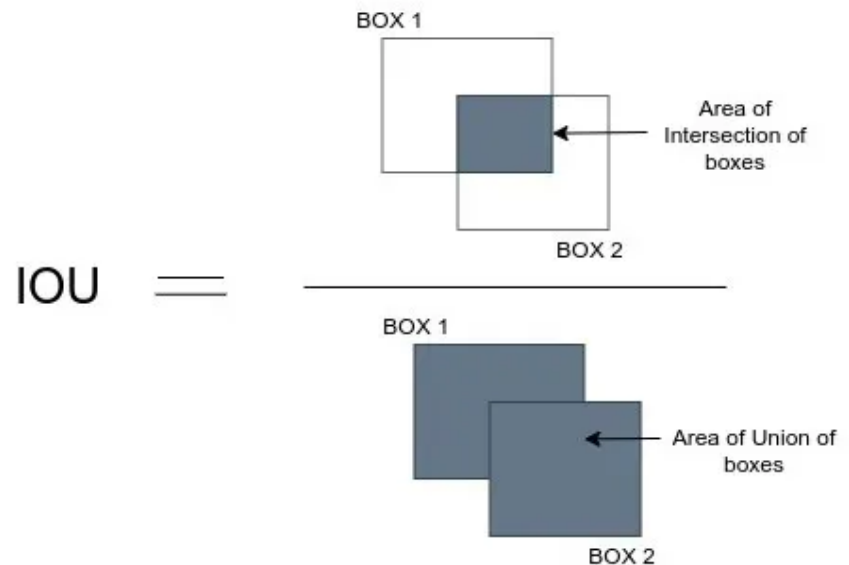
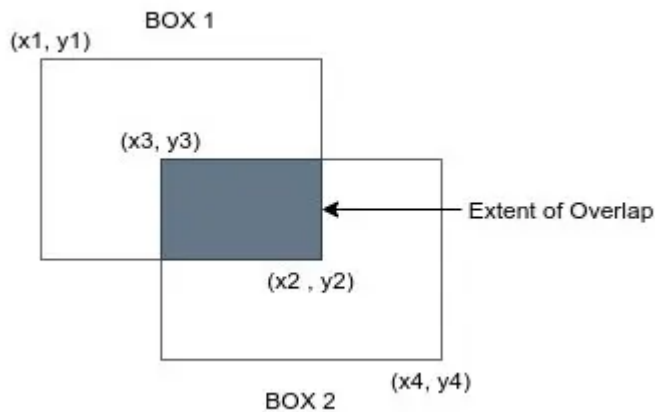
Even if the class is correct, the bounding box may be badly placed.

IoU tells us:

- “How accurate is the predicted box location?”

Let us assume that box 1 is represented by $[x1, y1, x2, y2]$, and box 2 is represented by $[x3, y3, x4, y4]$.

$$IOU = \frac{\text{Area of Intersection of two boxes}}{\text{Area of Union of two boxes}}$$



A detection is considered correct only if:

$$\text{IoU} > \text{Threshold}$$

Common threshold:

$$\text{IoU} \geq 0.5$$

Meaning:

- at least 50% overlap needed.

Given a Truth Box Area 100 the predicted box area is 120 and the overlap is 80. Find the Intersection over union.

$$\text{Union} = 100 + 120 - 80 = 140$$

$$\text{IoU} = 80 / 140 = 0.571\%$$

Threshold is 0.5

$$0.571 > 0.5$$

Detection is correct.

Non-Max Suppression (NMS)

Non-Max Suppression (NMS) is a technique used in object detection to remove **duplicate overlapping bounding boxes** for the same object.

Why NMS is Needed

Object detectors often predict:

- multiple boxes
- around the same object

Example:

A detector may predict 5 boxes for one dog.

Without NMS:

- multiple detections for same object.

With NMS:

- keep only the best box.

Working of NMS

- If there are many boxes for one object
- Step 1: Sort by Confidence
- Step 2: Select Highest Confidence Box
- Step 3: Compute IoU
- Step 4: Remove Highly Overlapping Boxes

Object Detection Algorithms

Object detection algorithms are used to:

- Detect objects**

- Classify objects**

- Locate objects using bounding boxes**

They are mainly divided into:

- Two-stage detectors (RCNN)

- Single-stage detectors

R-CNN (Region CNN)

Working of R-CNN

Step 1: Region Proposal

Generate about 2000 possible object regions using selective search.

Step 2: CNN Feature Extraction

Each region passed through CNN.

Step 3: Classification

Use SVM classifier.

Step 4: Bounding Box Regression

Improve box coordinates.

RCNN ALGORITHM

Input: Image

Output: Detected object classes and bounding boxes

1. Use Selective Search to generate region proposals (around 2000 regions).
2. For each region proposal:
 - a. Crop and warp the region to a fixed size (e.g., 224x224).
 - b. Pass the cropped region through a pre-trained CNN (e.g., AlexNet).
 - c. Extract the feature vector (e.g., 4096-dimensional).
3. For each feature vector:
 - a. Use a trained SVM classifier to predict the object class (dog, cat, car, etc.).
 - b. Use a trained Bounding Box Regressor to adjust the bounding box coordinates.
4. Apply Non-Maximum Suppression (NMS) to eliminate duplicate detections.
5. Output the final bounding boxes and corresponding object classes.

YOLO algorithm

You Only Look Once — one of the most famous and fastest object detection techniques.

What is YOLO?

YOLO is a **real-time object detection algorithm**.

It **predicts bounding boxes and class probabilities** at the same time.

Instead of **sliding window** or **region proposals** (like older methods), YOLO **looks at the entire image once** and detects all objects **in a single forward pass** of the neural network.

It treats **detection as a single regression problem** — directly from **image pixels** to **bounding box coordinates** and **class labels**.

YOLO Basic Workflow

Step Action

1. Divide the image into an **$S \times S$ grid** (example: 7x7 for YOLOv1)
2. Each grid cell predicts **B bounding boxes** (example: B=2) Each bounding box has: (x, y, w, h, confidence)
3. Each grid cell also predicts **C class probabilities** (for example, C=20 if 20 object classes)
4. Multiply confidence with class probabilities to get final scores
5. Apply **Non-Maximum Suppression (NMS)** to remove duplicate detections

YOLO Output Details

Each bounding box prediction consists of:

(x, y) → center coordinates relative to the grid cell (not the entire image)

(w, h) → width and height relative to the whole image

Confidence Score → how likely the box contains an object

Each grid cell also predicts:

Class probabilities → probability of each class **conditional** on the object existing.

Thus, final prediction per bounding box is:

Class Score = Confidence X Class Probability